

Proxy Creation Facilities: Enriching Proxy Construction for Pointer-Semantics Polymorphism

Document #: P3401R1 [[Latest](#)] [[Status](#)]
Date: 2025-10-06
Project: Programming Language C++
Audience: LEWGI
LEWG
Reply-to: Tian Liao
<tian.lt@outlook.com>
Mingxin Wang
<mingxwa@outlook.com>

Contents

- [1 History](#)
 - [1.1 Changes from P3401R0](#)
- [2 Introduction](#)
 - [2.1 Motivation and scope](#)
 - [2.1.1 Problems addressed](#)
 - [2.1.2 Non-goals](#)
 - [2.2 Impact on the standard](#)
- [3 Considerations and design decisions](#)
 - [3.1 Automatic vs explicit storage strategy](#)
 - [3.2 Unified acceptability concept](#)
 - [3.3 Shared ownership handle constraints](#)
 - [3.4 View creation](#)
 - [3.5 Freestanding rationale](#)
- [4 Technical specifications](#)
 - [4.1 Feature test macro](#)
 - [4.2 Header synopsis \(additions\)](#)
 - [4.3 Exposition-only helper types](#)

- 4.4 Concept `proxiable_target`
- 4.5 Concept `inplace_proxiable_target`
- 4.6 General constraints
- 4.7 Function templates `make_proxy_inplace`
- 4.8 Function templates `allocate_proxy`
- 4.9 Function templates `allocate_proxy_shared`
- 4.10 Function templates `make_proxy_shared`
- 4.11 Function templates `make_proxy`
- 4.12 Function template `make_proxy_view`
- 4.13 Exceptions
- 4.14 Complexity
- 4.15 Remarks
- 5 Summary

Abstract

This paper proposes standard library factory functions that construct `proxy<F>` (and `proxy_view<F>`) objects with clear ownership and storage semantics: adaptive value construction (`make_proxy`), forced in-place (`make_proxy_inplace`), allocator-supplied (`allocate_proxy`), compact shared ownership (`make_proxy_shared` / `allocate_proxy_shared`), and non-owning view creation (`make_proxy_view`). Two new concepts, `proxiable_target` and `inplace_proxiable_target`, provide uniform, diagnosable constraints on value acceptability for these creation paths. Centralizing these facilities eliminates duplicated ad hoc small-buffer wrappers, allocator holders, and reference-counted pointer types; it yields consistent diagnostics, enables trivial relocation optimization, and makes allocation and borrowing intent explicit.

§ 1 History

§ 1.1 Changes from P3401R0

- Added concept `proxiable_target` to distinguish any pointer-like type already modeling `proxiable` from a value type that can be wrapped as an acceptable target through an observing or owning witness.
- Strengthened template parameter constraints to require facade `F` where previously `class F` appeared.
- Added shared ownership creation (`allocate_proxy_shared`, `make_proxy_shared`) and non-owning view creation (`make_proxy_view`).

- Clarified freestanding status: value-only (`make_proxy_inplace`, `make_proxy_view`) remain available in freestanding; allocation-enabled functions are *freestanding-deleted*.
- Specified ill-formed conditions (diagnostic required) using `proxiable_target` / `inplace_proxiable_target` rather than substitution failures with less direct messaging.

§ 2 Introduction

This paper proposes to standardize a minimal, performance-oriented set of creation facilities for the pointer-semantics polymorphism facility (P3086: class template proxy and its facade model). The goal is to let users express (1) ownership intent, (2) storage strategy, and (3) allocator choice explicitly, without re-implementing small buffer optimization (SBO), reference counting handles, or view wrappers ad hoc. These factories are intentionally narrow: they neither expose user-visible control blocks nor introduce new abstraction layers beyond what is required to build an efficient pointer-like witness.

Two supporting concepts provide uniform, diagnosable constraints over value types passed to the factories:

- `proxiable_target<T, F>`: a value type that can be turned into some pointer-like witness acceptable to facade `F` (observing or owning path).
- `inplace_proxiable_target<T, F>`: a value type whose object can reside directly (in-place) within a witness subject to facade size/alignment and lifetime constraint levels.

Proposed additions (all in `<memory>`):

- Concepts: `proxiable_target`, `inplace_proxiable_target`.
- Value-based adaptive construction: `make_proxy` (decides in-place vs allocation).
- Forced in-place construction: `make_proxy_inplace` (compile-time rejection if impossible).
- Allocator supply: `allocate_proxy` (explicit allocator parameter, single object construction only).
- Shared ownership (compact pointer-sized control block semantics): `allocate_proxy_shared`, `make_proxy_shared`.
- Non-owning borrowing view: `make_proxy_view` producing `proxy_view<F>`.

§ 2.1 Motivation and scope

§ 2.1.1 Problems addressed

To use proxy effectively, users today must hand craft one or more of:

- An *adaptive* value factory deciding between in-place storage and heap allocation for a given facade (risk: duplicated heuristics, silent allocation differences across translation units).
- A *forced* in-place constructor expressing a hard performance contract (currently emulated by static assertions scattered through code).
- A *custom allocator* path that stores allocator state adjacent to the erased object without imposing an extra indirection or dynamic allocation for the allocator itself.
- A *compact shared ownership* handle (pointer-sized, cache-friendly, trivial relocation where possible) instead of layering `shared_ptr` (with a separately allocated control block whose shape is outside facade constraint reasoning).
- A *view* (borrow) wrapper that preserves facade semantics while preventing accidental ownership transfer.

Each local reinvention increases risk of ABI drift and subtle performance regressions (e.g. unintentional dynamic allocation where an in-place object would have satisfied the facade). Consolidating these patterns as standard factories concentrates optimization effort and yields consistent diagnostics.

§ 2.1.2 Non-goals

- Redefining facade composition or substitution semantics.
- Introducing polymorphic allocator support beyond supplying an allocator object to a single construction path.
- Standardizing builder / skill helper utilities (to be proposed separately when committee bandwidth allows).

§ 2.2 Impact on the standard

The proposal adds two concepts and six families of function templates to `<memory>`. No existing interfaces change. The feature test macro `__cpp_lib_proxy` is updated. No deprecations are introduced. ABI impact is limited to additional exported (or inline) templates; internal layout of proxy is unchanged.

Freestanding status:

- Provided: `make_proxy_inplace`, `make_proxy_view`, `proxiable_target`, `inplace_proxiable_target`.
- Freestanding-deleted: `make_proxy`, `allocate_proxy`, `make_proxy_shared`, `allocate_proxy_shared` (dynamic allocation / reference counting).

§ 3 Considerations and design decisions

§ 3.1 Automatic vs explicit storage strategy

`make_proxy` chooses between in-place storage and heap allocation at compile time using facade constraint parameters and target type traits. This mirrors existing practice in widely deployed implementations and shields users from reimplementing small buffer handles incorrectly. A separate `make_proxy_inplace` exists to force compile-time rejection when in-place storage is not possible, providing a clear intent signal for latency-sensitive code that wants to rule out allocation.

§ 3.2 Unified acceptability concept

Before introducing `proxiable_target`, value-type acceptability errors surfaced indirectly through template instantiations of helper wrappers. `proxiable_target<T, F>` declaratively states: a value of type `T` can be made into a pointer-like witness acceptable to facade `F` via an observing or owning handle. This improves diagnostics (ill-formed with required messages) and enables consistent constraints across allocating, shared, and observing creation functions.

§ 3.3 Shared ownership handle constraints

The shared ownership creation functions employ exposition-only *strong-compact-ptr* / *weak-compact-ptr* types whose size and alignment never exceed those of a raw pointer. This ensures facade layout constraints remain satisfied when users intend reference counting but still desire compact handles (important for cache-sensitive polymorphic graphs). The deleter used by the internal shared handle is a concrete, non-erased deleter selected at compile time; type-erasing the deleter would add an avoidable level of indirection and storage overhead. Destruction polymorphism (invoking the correct destruction behavior for the erased *value type*) is already provided by proxy itself, so erasing the deleter yields no additional semantic power.

§ 3.4 View creation

`make_proxy_view` produces a `proxy_view<F>` that borrows a referenced object. Using a dedicated function clarifies that no ownership transfer or allocation occurs, and allows a direct constraint based on `proxiable_target`. The selected witness (*observer-ptr*) is trivial and can always be trivially copied and relocated, preserving facade constraint levels.

§ 3.5 Freestanding rationale

In-place and view construction are retained in freestanding because they can be implemented without dynamic allocation and do not require reference counting machinery. Allocation and shared ownership rely on dynamic storage and are therefore *freestanding-deleted*, matching the base proxy approach to allocator-dependent facilities.

§ 4 Technical specifications

This section contains proposed wording additions to `<memory>`. Exposition-only entities have no linkage.

§ 4.1 Feature test macro

In `[version.syn]`, update:

```
#define __cpp_lib_proxy YYYYMMML // also in <memory>
```

The value is updated to the date of adoption of this paper.

§ 4.2 Header synopsis (additions)

```
namespace std {

    // concepts
    template<class T, class F>
        concept proxiable_target = see below;
    template <class T, class F>
        concept inplace_proxiable_target = see below;

    // value-based creation (freestanding-deleted)
    template<facade F, class T, class... Args>
        proxy<F> make_proxy(Args&&... args); // freestanding-deleted
    template<facade F, class T, class U, class... Args>
        proxy<F> make_proxy(initializer_list<U> il, Args&&... args); // free-
            standing-deleted
    template<facade F, class T>
        proxy<F> make_proxy(T&& value); // freestanding-deleted

    // allocator-supplied creation (freestanding-deleted)
    template<facade F, class T, class Alloc, class... Args>
        proxy<F> allocate_proxy(const Alloc& alloc, Args&&... args); // free-
            standing-deleted
    template<facade F, class T, class Alloc, class U, class... Args>
        proxy<F> allocate_proxy(const Alloc& alloc, initializer_list<U> il,
            Args&&... args); // freestanding-deleted
    template<facade F, class Alloc, class T>
        proxy<F> allocate_proxy(const Alloc& alloc, T&& value); // freestand-
            ing-deleted

    // in-place forced creation (freestanding)
    template<facade F, class T, class... Args>
        proxy<F> make_proxy_inplace(Args&&... args) noexcept(is_nothrow_con-
            structible_v<T, Args...>)
            requires(is_constructible_v<T, Args...>);
    template<facade F, class T, class U, class... Args>
        proxy<F> make_proxy_inplace(initializer_list<U> il, Args&&... args)
            noexcept(is_nothrow_constructible_v<T, initializer_list<U>&,
                Args...>)
            requires(is_constructible_v<T, initializer_list<U>&, Args...>);
    template<facade F, class T>
        proxy<F> make_proxy_inplace(T&& value)
            noexcept(is_nothrow_constructible_v<decay_t<T>, T>)
```

```

    requires(is_constructible_v<decay_t<T>, T>);

    // shared ownership creation (freestanding-deleted)
    template<facade F, class T, class Alloc, class... Args>
        proxy<F> allocate_proxy_shared(const Alloc& alloc, Args&&... args); //
        freestanding-deleted
    template<facade F, class T, class Alloc, class U, class... Args>
        proxy<F> allocate_proxy_shared(const Alloc& alloc, initializ-
        er_list<U> il, Args&&... args); // freestanding-deleted
    template<facade F, class Alloc, class T>
        proxy<F> allocate_proxy_shared(const Alloc& alloc, T&& value); //
        freestanding-deleted

    template<facade F, class T, class... Args>
        proxy<F> make_proxy_shared(Args&&... args); // freestanding-deleted
    template<facade F, class T, class U, class... Args>
        proxy<F> make_proxy_shared(initializer_list<U> il, Args&&... args); //
        freestanding-deleted
    template<facade F, class T>
        proxy<F> make_proxy_shared(T&& value); // freestanding-deleted

    // view creation (freestanding)
    template<facade F, class T>
        proxy_view<F> make_proxy_view(T& value) noexcept;
}

```

§ 4.3 Exposition-only helper types

The following exposition-only types are used in the specifications below:

- *inplace_ptr*: same size and alignment as *T*; contains storage for a *T*; provides `operator*` with matching cv/ref qualifications; no empty state.
- *allocated_ptr<T, Alloc>*: manages dynamically allocated storage for a single *T* constructed with *Alloc*; provides `operator*`.
- *strong-compact_ptr<T, Alloc>*: reference-counted owning handle; size and alignment not greater than those of a raw pointer; conditionally convertible to *weak-compact_ptr<T, Alloc>*.
- *weak-compact_ptr<T, Alloc>*: non-owning handle referencing the same control block as a *strong-compact_ptr*.
- *observer_ptr*: trivially copyable wrapper of a raw *T**; provides `operator*`.

§ 4.4 Concept `proxiable_target`

```

template<class T, class F>
concept proxiable_target = see below;

```

`proxiable_target<T, F>` is equivalent to `proxiable<observer_ptr, F>`.

§ 4.5 Concept `inplace_proxiable_target`

```
template<class T, class F>
concept inplace_proxiable_target = see below;
```

`inplace_proxiable_target<T, F>` is equivalent to `proxiable<inplace_ptr, F>`.

§ 4.6 General constraints

For each function template family (`make_proxy`, `allocate_proxy`, `allocate_proxy_shared`, `make_proxy_shared`, `make_proxy_view`, `make_proxy_inplace`) with declarations numbered (1–3):

- If a function forms a witness from a value of (possibly cv/ref-qualified) type `T` other than via in-place construction, the program is ill-formed if `!proxiable_target<decay_t<T>, F>`.
- For in-place creation the program is ill-formed if `!inplace_proxiable_target<decay_t<T>, F>`.
- For view creation the program is ill-formed if `!proxiable_target<T, F>`.

Ill-formed conditions require a diagnostic; they do not participate in overload resolution via substitution failure.

§ 4.7 Function templates `make_proxy_inplace`

```
// (1)
template<facade F, class T, class... Args>
proxy<F> make_proxy_inplace(Args&&... args)
    noexcept(is_nothrow_constructible_v<T, Args...>)
    requires(is_constructible_v<T, Args...>);

// (2)
template<facade F, class T, class U, class... Args>
proxy<F> make_proxy_inplace(initializer_list<U> il, Args&&... args)
    noexcept(is_nothrow_constructible_v<T, initializer_list<U>&, Args...>)
    requires(is_constructible_v<T, initializer_list<U>&, Args...>);

// (3)
template<facade F, class T>
proxy<F> make_proxy_inplace(T&& value)
    noexcept(is_nothrow_constructible_v<decay_t<T>, T>)
    requires(is_constructible_v<decay_t<T>, T>);
```

Effects: (1) Creates a `proxy<F>` containing a value `p` of type `inplace_ptr` where `*p` is direct-non-list-initialized with `std::forward<Args>(args)...`. (2) Same but with `il` as the first argument to the construction of `*p`. (3) Same as (1) with `T` replaced by `decay_t<T>` and the single argument `std::forward<T>(value)`.

Remarks: No dynamic allocation is performed. The functions are `noexcept` as specified.

§ 4.8 Function templates `allocate_proxy`

```

// (1)
template<facade F, class T, class Alloc, class... Args>
proxy<F> allocate_proxy(const Alloc& alloc, Args&&... args); // freestanding-deleted

// (2)
template<facade F, class T, class Alloc, class U, class... Args>
proxy<F> allocate_proxy(const Alloc& alloc, initializer_list<U> il,
    Args&&... args); // freestanding-deleted

// (3)
template<facade F, class Alloc, class T>
proxy<F> allocate_proxy(const Alloc& alloc, T&& value); // freestanding-deleted

```

Effects: (1) Creates a proxy<F> containing a value p of type *allocated_ptr<T, Alloc>* where *p is direct-non-list-initialized with `std::forward<Args>(args)...`. (2) Same with il, `std::forward<Args>(args)...`. (3) Same with T replaced by `decay_t<T>` and argument `std::forward<T>(value)`.

Throws: Any exception thrown by allocation or construction of T. May allocate additional storage for a copy of alloc if required by the facade constraints.

§ 4.9 Function templates `allocate_proxy_shared`

```

// (1)
template<facade F, class T, class Alloc, class... Args>
proxy<F> allocate_proxy_shared(const Alloc& alloc, Args&&... args); // freestanding-deleted

// (2)
template<facade F, class T, class Alloc, class U, class... Args>
proxy<F> allocate_proxy_shared(const Alloc& alloc, initializer_list<U> il,
    Args&&... args); // freestanding-deleted

// (3)
template<facade F, class Alloc, class T>
proxy<F> allocate_proxy_shared(const Alloc& alloc, T&& value); // freestanding-deleted

```

Effects: As for `allocate_proxy` but the contained value is of type *strong_compact_ptr<X, Alloc>* (where X is T or `decay_t<T>`). The construction of X and any control block are treated as a single operation for exception safety.

Throws: Any exception thrown by allocation or by construction of X.

§ 4.10 Function templates `make_proxy_shared`

```

// (1)
template<facade F, class T, class... Args>
proxy<F> make_proxy_shared(Args&&... args); // freestanding-deleted

// (2)
template<facade F, class T, class U, class... Args>
proxy<F> make_proxy_shared(initializer_list<U> il, Args&&... args); // freestanding-deleted

// (3)

```

```
template<facade F, class T>
proxy<F> make_proxy_shared(T&& value); // freestanding-deleted
```

Effects: (1) Equivalent to `return allocate_proxy_shared<F, T>(std::allocator<void>{}, std::forward<Args>(args)...);` (2) Equivalent to the same with `il`. (3) Equivalent to (1) with `T` replaced by `decay_t<T>`.

§ 4.11 Function templates `make_proxy`

The definitions of `make_proxy` use an exposition-only function template:

```
template<facade F, class T, class... Args>
proxy<F> __make_proxy_internal(Args&&... args) {
    if constexpr (inplace_proxiable_target<T, F>) {
        return make_proxy_inplace<F, T>(std::forward<Args>(args)...);
    } else {
        return allocate_proxy<F, T>(std::allocator<void>{}, std::forward<Args>(args)...);
    }
}
```

```
// (1)
template<facade F, class T, class... Args>
proxy<F> make_proxy(Args&&... args); // freestanding-deleted
// (2)
template<facade F, class T, class U, class... Args>
proxy<F> make_proxy(initializer_list<U> il, Args&&... args); // freestanding-deleted
// (3)
template<facade F, class T>
proxy<F> make_proxy(T&& value); // freestanding-deleted
```

Effects: (1) Equivalent to `return __make_proxy_internal<F, T>(std::forward<Args>(args)...);` (2) Equivalent to `return __make_proxy_internal<F, T>(il, std::forward<Args>(args)...);` (3) Equivalent to `return __make_proxy_internal<F, decay_t<T>>(std::forward<T>(value));`.

Throws: Any exception thrown by allocation or construction of `T`.

§ 4.12 Function template `make_proxy_view`

```
template<facade F, class T>
proxy_view<F> make_proxy_view(T& value) noexcept;
```

Effects: Creates a `proxy_view<F>` containing a value `p` of type *observer_ptr* where `*p` is direct-non-list-initialized with `&value`.

Remarks: No allocation; always `noexcept`.

§ 4.13 Exceptions

In-place and view creation functions only propagate exceptions thrown by construction of the target value (view creation cannot throw). All other creation functions may additionally throw due to allocation failure.

§ 4.14 Complexity

Each function has $O(1)$ time complexity with respect to the number of forwarded arguments. Reference counting initialization for shared forms is $O(1)$.

§ 4.15 Remarks

Implementations may perform trivial relocation when the facade's relocatability constraint level permits and the witness type satisfies the corresponding requirements.

§ 5 Summary

The proposed factories supply:

- A single adaptive entry point (`make_proxy`) enabling SBO without user code.
- An explicit contract form (`make_proxy_inplace`) to reject accidental allocation.
- Allocator and compact shared ownership creation paths designed for predictable layout and trivial relocation opportunities.
- A lightweight borrowing view (`make_proxy_view`) clarifying lifetime without re-specifying facade operations.
- Two concepts delivering concise diagnostics around value acceptability.

Centralizing these patterns in the standard library removes duplicated bespoke wrappers, improves portability of optimizations, and clarifies performance contracts around allocation and ownership for pointer-semantics polymorphism.